

Learning Management System - Project Goal

This Learning Management System project will be a complete full stack web application built using Python, Django, PostgreSQL, Django REST Framework, Email Services, Docker, and GitHub Actions CI. The objective is to deliver one integrated platform that demonstrates every topic covered in training through a real academic and industry relevant LMS system. The final submission must include a working web application where students can enroll in courses, instructors can manage content, and admins can control the entire platform.

The project must include a User and Role Management Module where users can register, login, and access the system based on roles such as Student, Instructor, and Admin. Authentication and authorization must be implemented using Django's built in security system. Profile management must be supported, and password reset with email confirmation must be included using Django Email Service.

The Course Management Module must allow instructors and admins to create, update, and delete courses. Each course must contain structured details such as title, description, category, level, and slug based URLs for SEO friendly navigation. Courses must be displayed through Jinja2 templates with {Tailwind integrated responsive UI} (Optional). Students must be able to browse courses with filters such as category and difficulty.

The Enrollment Module must allow students to enroll into courses, track enrolled courses, and view progress. Enrollment data must be stored in PostgreSQL with proper ORM relationships between users and courses. Exception handling must manage cases like duplicate enrollments or invalid course access.

The Content Delivery Module must support adding lessons, video links, notes, and downloadable resources inside each course. Instructors must manage lesson sequences, and students must be able to access content step by step. Iterators or generators can be used for lesson pagination and efficient delivery of course modules.

The Assignment and Quiz Module must allow instructors to create assignments or quizzes, and students must submit responses. Submissions must be stored

securely, and evaluation status must be tracked. Python OOP concepts must be applied through service classes such as AssignmentManager or QuizEvaluator, using abstraction and encapsulation principles.

The Admin Dashboard Module must provide complete control over users, courses, enrollments, assignments, and platform activity through Django Admin Panel customization. Admins must be able to monitor course statistics, student participation, and instructor activity.

The LMS must include a Notification and Email Module where automated emails are triggered for signup confirmation, course enrollment confirmation, assignment deadlines, and instructor announcements. Email templates must be structured professionally and integrated into the workflow.

A Django REST Framework API Module must expose endpoints for LMS functionality such as course listing, enrollment actions, lesson access, and assignment submissions. Serializers must be created for all models, and ViewSets with routers must provide REST APIs for integration with mobile apps or external systems.

PostgreSQL must be used as the primary database, integrated through Django ORM. Database creation and management must be demonstrated using pgAdmin or DBeaver. CRUD operations should also be practiced through PostgreSQL command line queries to show database proficiency.

The project must be deployment ready through Docker containerisation. The submission must include a Dockerfile for the Django app, a docker-compose.yml setup connecting Django with PostgreSQL, and environment variable based configuration for production readiness.

A complete CI pipeline must be implemented using GitHub Actions. The workflow must automate linting, running tests, and validating Docker builds on every push. The repository must contain a working .github/workflows/ci.yml file with successful pipeline execution proof.

Python programming concepts must be clearly integrated across the project, including functions, data structures, OOP design, static methods for utilities, shallow vs deep copy demonstrations in course content handling, and structured exception handling. Practice inspired utilities such as split join or swapcase can be included in helper modules for additional Python coverage.

The final LMS submission must include complete source code on GitHub, a professional README with setup and execution steps, database schema diagram, API documentation, UI screenshots, Docker deployment guide, CI

pipeline proof, and a fully functional LMS web platform demonstrating backend development, database integration, REST APIs, DevOps practices, and strong Python foundations.

Project Goal

Build a full stack LMS using Python, Django, DRF, PostgreSQL, Docker, and CI.

This project must demonstrate every topic you covered in training.

Core Modules to Build

1. User Authentication and Roles

Deliverables

- Signup and Login system using Django ORM
- Role based access control
- Roles
- Student
- Instructor
- Admin

Requirements

- Authentication using Django built in auth
- Authorization using permissions and groups
- Secure password handling
- Profile management

2. Course Management Module

Deliverables

- Create, update, delete courses
- Course listing with search and filters
- Course detail page with syllabus

Requirements

- Use Models, Views, Templates
- Slug based course URLs

Example

```
/course/python-for-beginners/
```

3. Lesson and Content Delivery

Deliverables

- Add lessons under courses
- Upload resources like PDFs or links
- Track lesson completion

Requirements

- Foreign key relationships
- Django Admin management
- Template rendering using Jinja2

4. Student Enrollment System

Deliverables

- Students can enroll in courses
- Instructor can view enrolled students
- Enrollment dashboard

Requirements

- Many to Many relationships
- ORM queries for analytics

Example

```
Total enrolled students per course
```

5. Assignment and Quiz Module

Deliverables

- Instructor creates assignments

- Students submit answers
- Quiz with MCQ support

Requirements

- Exception handling for invalid submissions
- OOP based evaluation logic
- Use generators for quiz question loading

6. Email Notification System

Deliverables

- Email on successful signup
- Email on course enrollment
- Email reminder for pending assignments

Requirements

- Django Email backend integration
- Custom email templates

7. REST API Layer using DRF

Deliverables

- API for Courses
- API for Lessons
- API for Enrollments
- API for Assignments

Requirements

- ModelSerializers
- API authentication using Token or JWT
- Permissions for Student vs Instructor

Example Endpoints

```
GET /api/courses/
```

```
POST /api/enroll/
```

```
GET /api/my-courses/
```

8. Admin Dashboard and Database Integration

Deliverables

- Admin panel for full LMS control
- PostgreSQL integration
- Database setup through pgAdmin or DBeaver

Requirements

- PostgreSQL as production DB
- Raw SQL CRUD through command line
- ORM based queries in Django

9. Frontend UI with Tailwind

Deliverables

- Responsive LMS pages
- Course cards
- Student dashboard
- Instructor dashboard

Requirements

- Tailwind integration in Django templates
- Clean layout for usability

10. CI Pipeline using GitHub Actions

Deliverables

- Automated testing workflow
- Lint checks
- Deployment ready pipeline

Requirements

- Run Django tests on every push
- Check Docker build success

Example Workflow Steps

- Install dependencies
- Run migrations
- Run unit tests

11. Containerization using Docker

Deliverables

- Dockerfile for Django app
- Docker Compose for PostgreSQL + App

Requirements

- One command setup

```
docker-compose up
```

12. Python Concepts Integration and LMS

Deliverables Mapping

Python Topic → LMS Usage

- Functions → Business logic helpers
- Lists, Sets, Tuples → Course data handling
- Dictionaries → Quiz answer mapping
- OOP → Course, Student, Instructor classes
- Inheritance → User role classes
- Polymorphism → Different content types
- Abstraction → Base models
- Encapsulation → Private methods in services
- Exception handling → Enrollment errors
- Iterators vs Generators → Lazy loading lessons
- eval() → Restricted quiz evaluation

- Shallow vs Deep Copy → Course duplication feature
- Static and Class methods → Utility functions

Final Project Deliverables Checklist

Code Deliverables

- Complete Django LMS project
- DRF APIs implemented
- PostgreSQL database integration
- Dockerized setup
- CI pipeline configured

Documentation Deliverables

- README with setup steps
- API documentation
- Database schema diagram
- Module wise explanation

Deployment Deliverables

- Docker Compose production ready build
- GitHub Actions workflow passing

Suggested Mini Features for Strong Impact

- Course progress tracker
- Certificate generation after completion
- Instructor analytics dashboard
- Search and pagination in APIs

Folder Structure

```
lms_project/  
  manage.py  
  requirements.txt  
  README.md  
  Dockerfile
```

```
docker-compose.yml
.env
.gitignore
.github/workflows/
lms_project/
apps/
lessons/
enrollments/
quizzes/
notifications/
static/
```